

Genetic Algorithm

TSP & CartPole-v1

BIAI — Progress Report

Piotr Krupiński · Jeremi Szczotka

Politechnika Śląska · 2026

github.com/piotrek1459/biai-genetic-cartpole

Selected Topics

Task 1 — TSP

Traveling Salesman Problem

Find the shortest route visiting all cities exactly once and returning to the start.

- Classic combinatorial optimisation
- Chromosome: **permutation** of city indices
- Fitness: $1 / (\text{distance} + \epsilon)$

Task 2 — CartPole-v1

Gymnasium reinforcement environment

Keep a pole balanced on a moving cart by pushing left or right.

- Control / policy learning task
- Chromosome: **real-valued** weight vector
- Fitness: mean episode reward

Both problems are fundamentally different — TSP requires permutation operators, CartPole requires real-valued operators. One codebase handles both.

Project Goals

1. Implement a **generic GA framework** reusable across problem types
2. Implement all core **genetic operators** from scratch:
 - Selection: tournament, roulette
 - Crossover: OX, PMX (permutation) · uniform, arithmetic (real-valued)
 - Mutation: swap, inversion, scramble (permutation) · Gaussian (real-valued)
3. **Solve TSP** for 20 cities — minimise total route distance
4. **Solve CartPole** — learn a linear policy that achieves max reward (500)
5. **Compare hyperparameters** — operators, population size, mutation strength
6. Produce visualisations, comparison plots, and an agent video

Plan of Work

Phase	Task	Status
1	Repository scaffold, <code>requirements.txt</code> , <code>virtualenv</code>	✓ Done
2	Common operators — <code>selection.py</code> , <code>mutation.py</code> , <code>crossover.py</code>	✓ Done
3	Generic <code>GeneticAlgorithm</code> class (<code>ga_base.py</code>)	✓ Done
4	Task 1: TSP — training, visualisation, experiments	✓ Done
5	Task 2: CartPole — policy, training, agent video, experiments	✓ Done
6	README, <code>.gitignore</code> , documentation, progress report	✓ Done

All phases complete. Both algorithms run end-to-end and produce results.

Implementation Overview

```
src/  
  common/  
    ga_base.py      ← GeneticAlgorithm class  
    selection.py    ← tournament, roulette  
    crossover.py    ← 0X, PMX, uniform, arithmetic  
    mutation.py     ← swap, inversion, scramble,  
                     gaussian  
  
  task1_tsp/  
    tsp_ga.py       ← CONFIG + run_tsp()  
    experiments.py  ← operator & population runs  
  
  task2_cartpole/  
    cartpole_policy.py ← LinearPolicy  
    train_ga.py      ← CONFIG + run_cartpole()  
    evaluate.py      ← record agent as MP4  
    experiments.py   ← sigma, pop, episodes
```

Design principle:

The `GeneticAlgorithm` class is **fully generic** — it receives operators as callables and

GA Core — Genetic Operators

Selection

Method	Key property
Tournament ($k=4$)	Tunable pressure via k
Roulette wheel	Proportional to fitness

Crossover

Operator	Type
OX — Order Crossover	Permutation
PMX — Partially Mapped	Permutation
Uniform	Real-valued
Arithmetic blend	Real-valued

Mutation

Operator	Type
Swap	Permutation
Inversion (<i>default TSP</i>)	Permutation
Scramble	Permutation
Gaussian $N(0, \sigma)$	Real-valued

Elitism

Top $n_{\text{elite}} = 2$ individuals are copied unchanged into the next generation, preventing loss of the best solution found so far

Task 1: TSP — Algorithm

Chromosome: permutation of city indices, e.g. [2, 0, 4, 1, 3]

→ route: city 2 → city 0 → city 4 → city 1 → city 3 → city 2

Fitness:

$$f(\text{route}) = \frac{1}{\text{total distance} + \varepsilon}$$

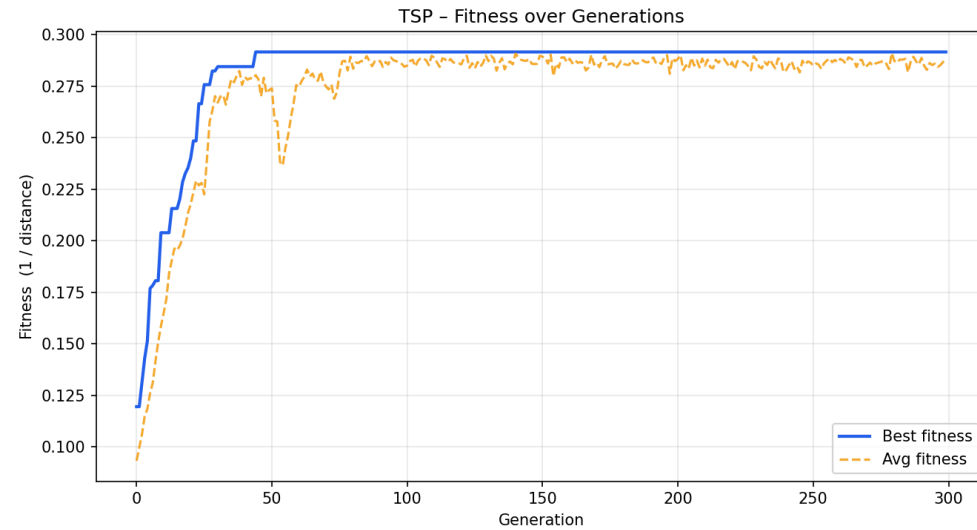
Maximising fitness $\equiv$ minimising total Euclidean round-trip distance.

Default configuration:

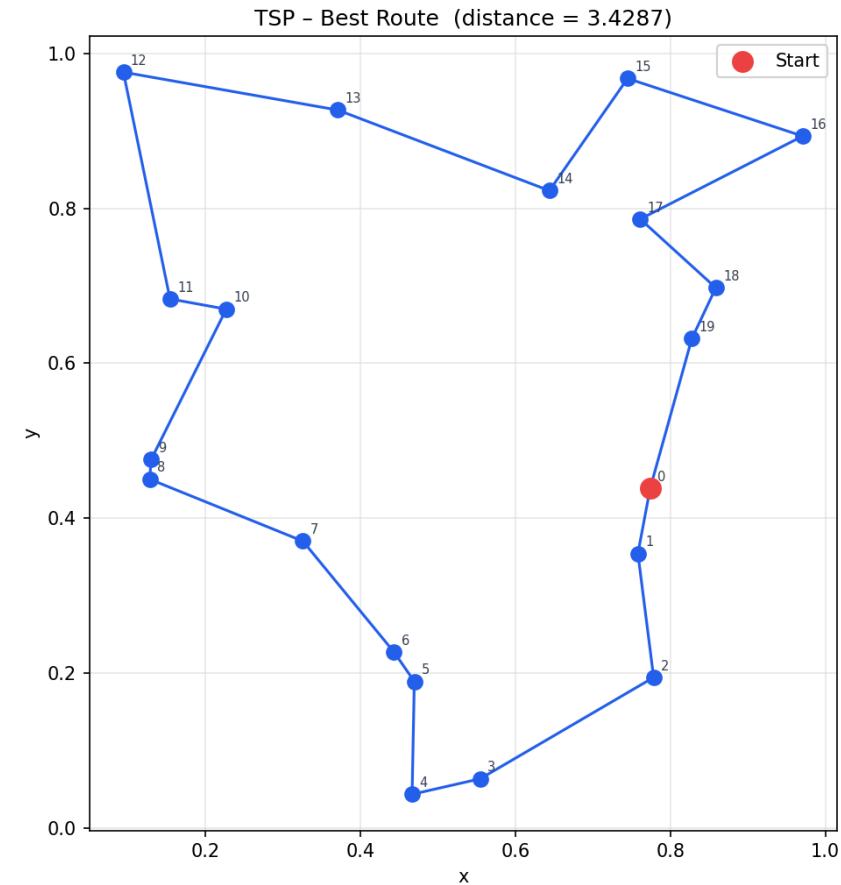
pop	generations	crossover	mutation	elites	tournament k
80	300	OX	Inversion, rate=0.10	2	4

Cities: 20 random points in $[0, 1]^2$ · Theoretical optimum ≈ 3.1

Task 1: TSP — Results



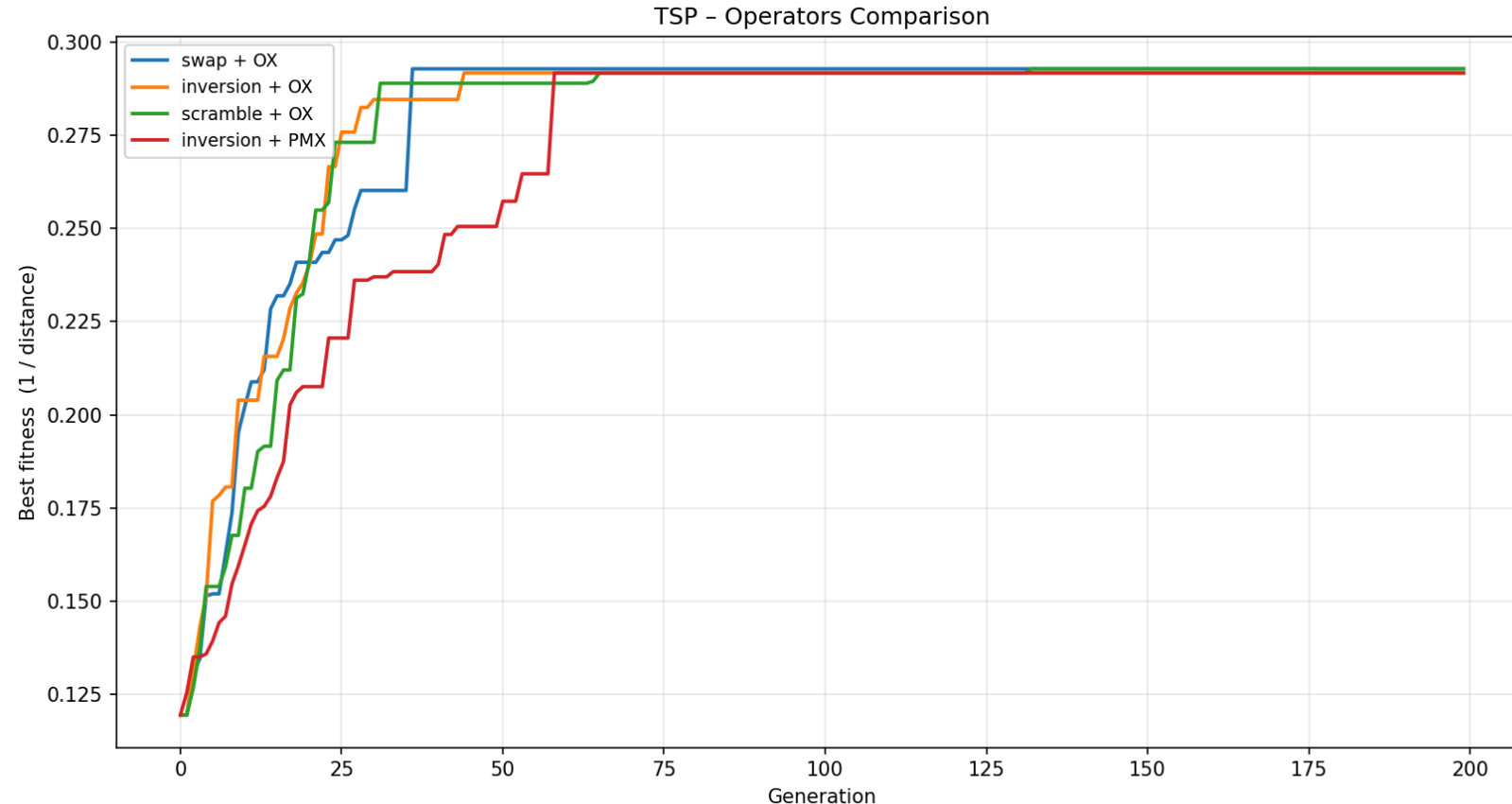
Fitness converges quickly in the first ~50 generations and stabilises around the near-optimal solution.



Best route distance: 3.43

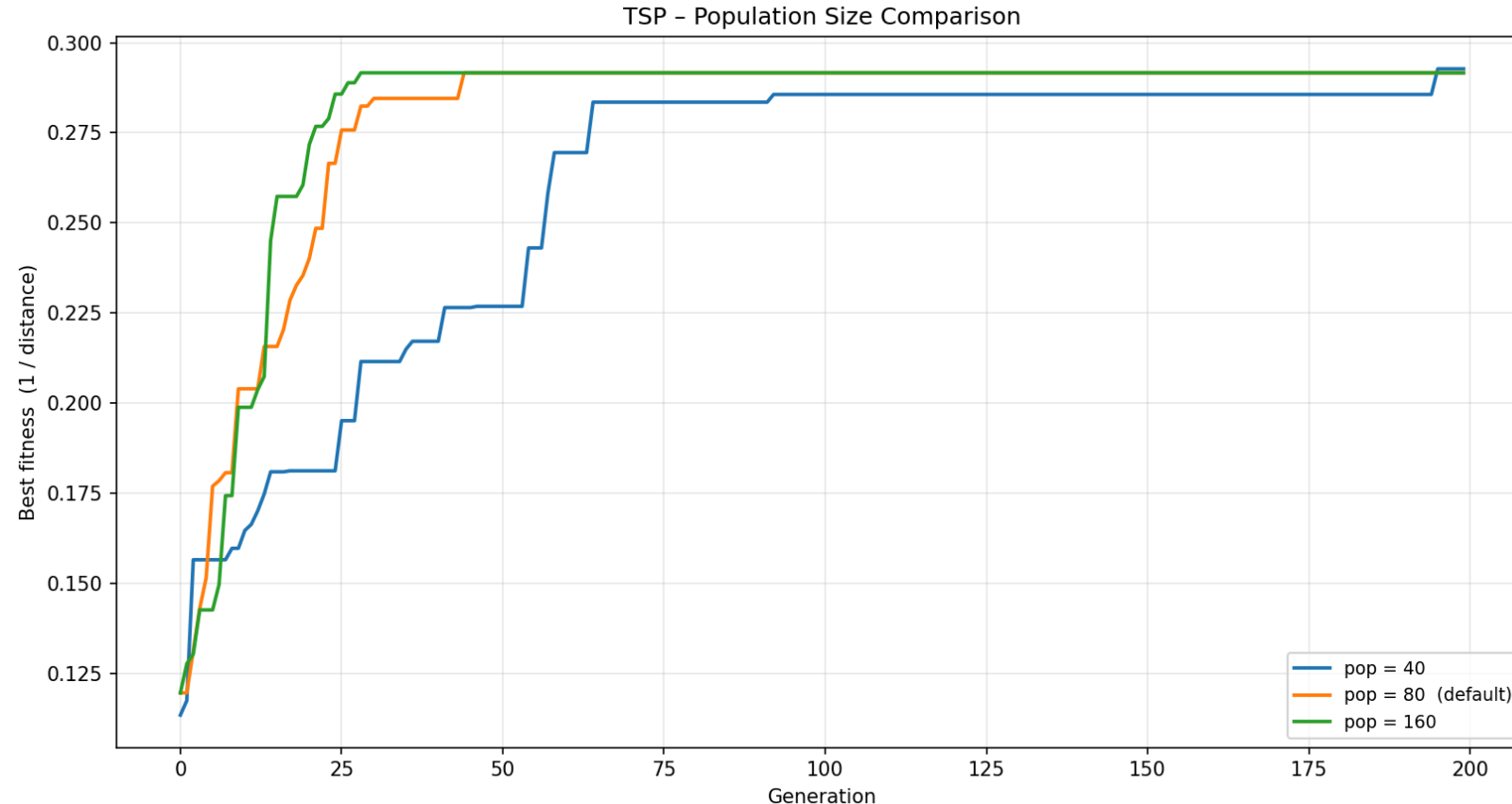
Theoretical optimum ≈ 3.1 — within ~10%

Task 1: TSP — Operator Comparison



All four operator combinations converge to a similar quality. **Inversion+OX** is the default choice — good balance of exploration and exploitation.

Task 1: TSP — Population Size Comparison



All population sizes reach similar final fitness. Larger populations show more stable convergence curves but require more computation per generation.

Task 2: CartPole — Algorithm

Environment: 4D observation [cart_pos, cart_vel, pole_angle, pole_angular_vel]

Action: push left (0) or push right (1) · **Max reward: 500**

Linear policy (chromosome = 5 real numbers):

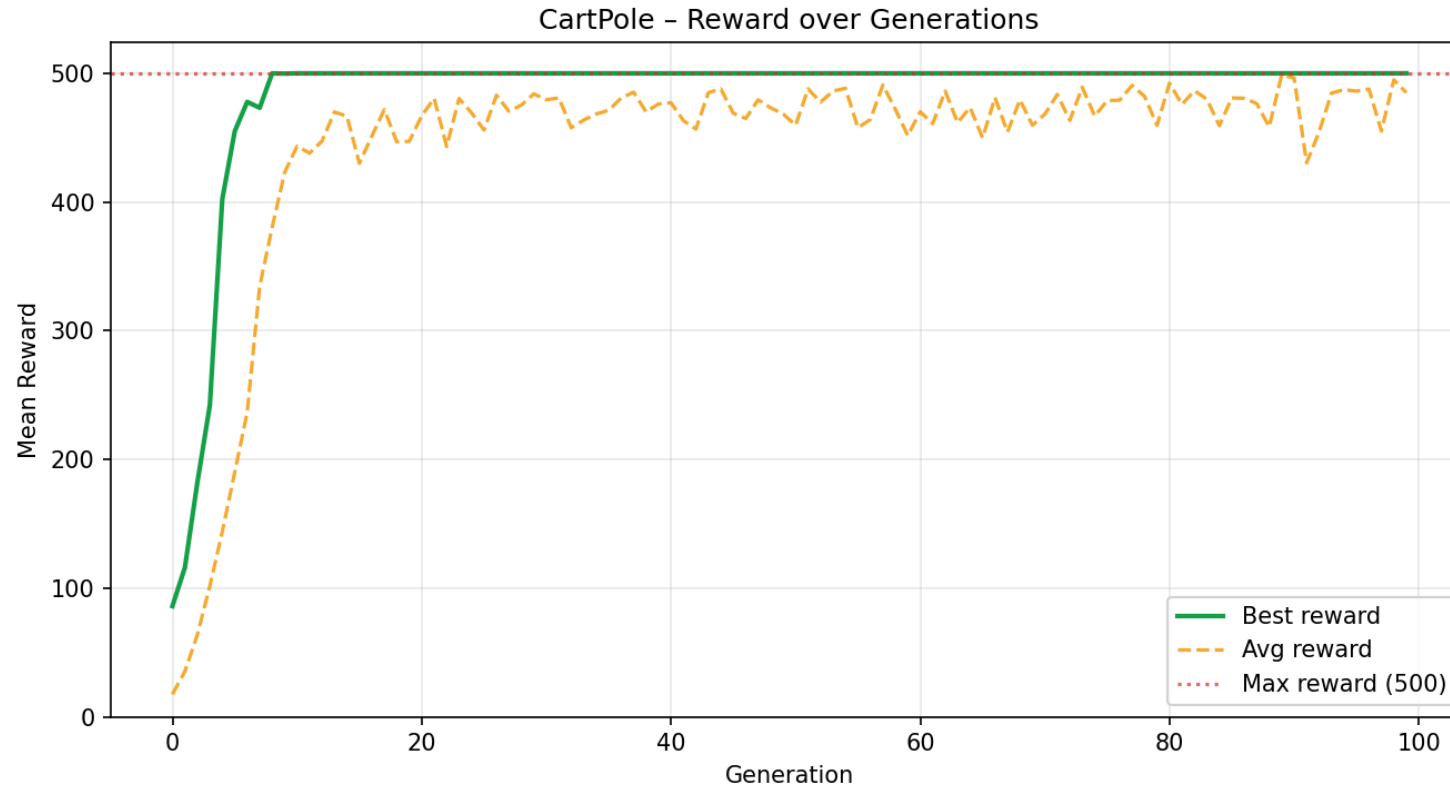
$$\text{action} = \begin{cases} 1 & \text{if } w_0 \cdot \text{pos} + w_1 \cdot \text{vel} + w_2 \cdot \theta + w_3 \cdot \dot{\theta} + b > 0 \\ 0 & \text{otherwise} \end{cases}$$

Fitness: mean total reward over 5 independent episodes.

Default configuration:

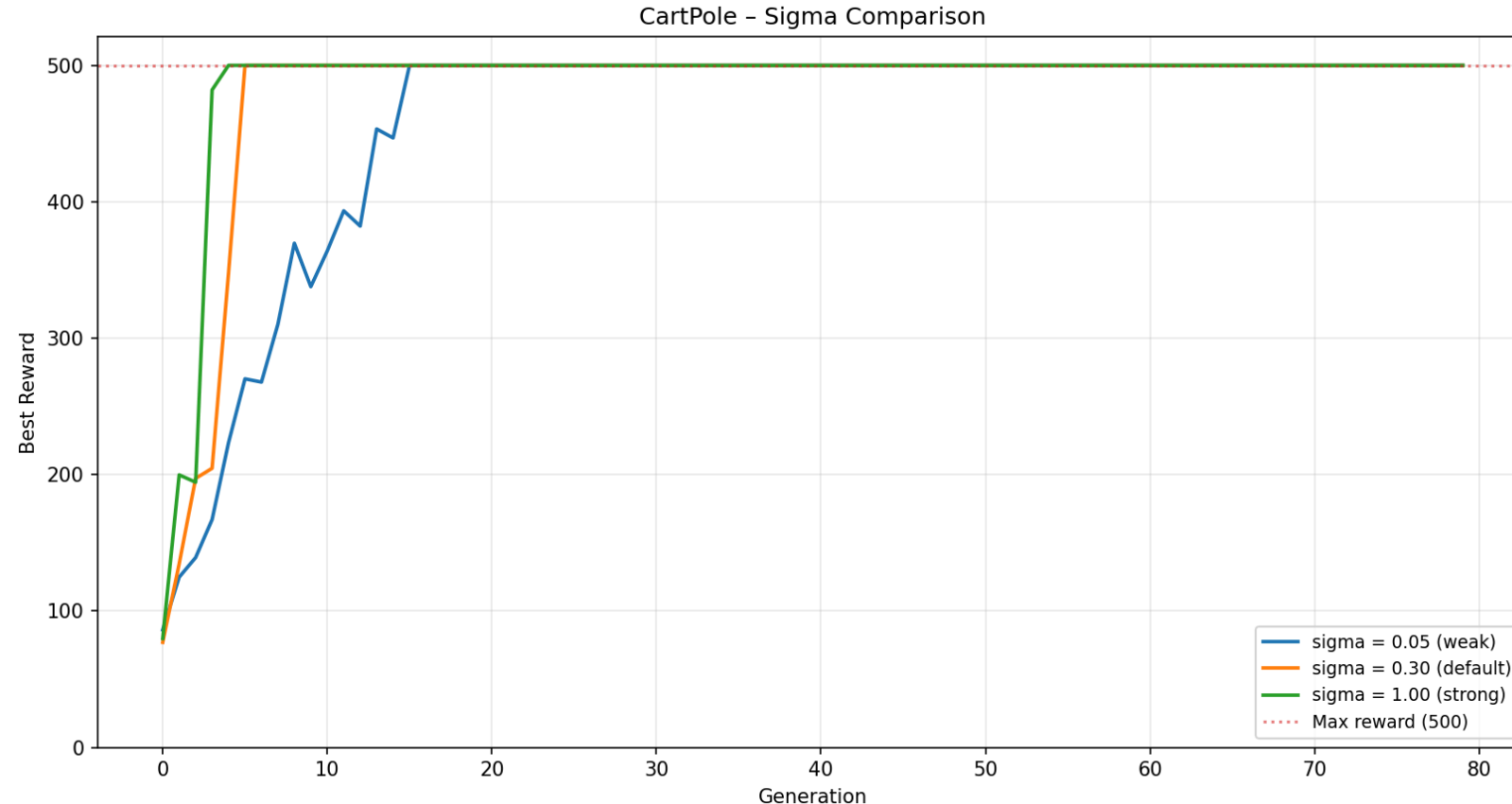
pop	generations	crossover	mutation	elites	episodes/eval
50	100	Uniform	Gaussian $\sigma=0.3$, rate=0.10	2	5

Task 2: CartPole — Results



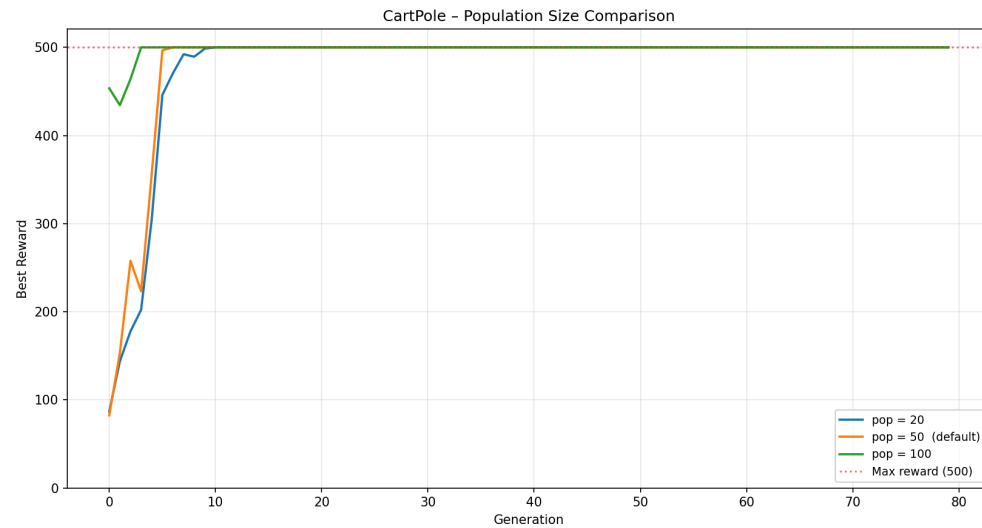
Maximum reward (500) reached at generation 10. The linear policy is sufficient to fully solve CartPole-v1. Agent video saved to `results/task2/agent.mp4`.

Task 2: CartPole — Sigma Comparison

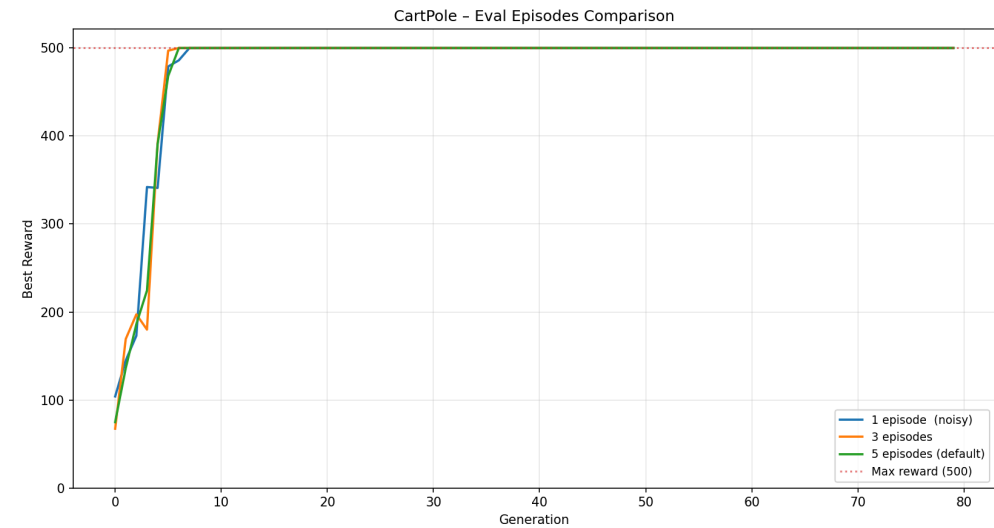


All three sigma values eventually reach reward 500. Smaller σ converges more smoothly; larger σ shows higher variance but can escape local optima faster.

Task 2: CartPole — Population & Episodes Comparison



Larger populations start with better initial diversity — `pop=100` hits reward 500 within the first few generations.



More evaluation episodes reduce fitness noise, leading to smoother and more reliable convergence.

Conclusions

Task 1 — TSP

- GA reduces route distance from ~6 (random) to **3.43** — within 10% of the theoretical optimum (~3.1)
- All operator combinations perform similarly; **OX + inversion** is the most consistent choice

Task 2 — CartPole

- GA discovers a working **linear policy** in just **10 generations**
- Best agent achieves **reward = 500/500** every episode (maximum possible)
- CartPole-v1 is linearly solvable — a 5-parameter chromosome is sufficient

Key Takeaway

A single generic GA framework with **pluggable operators** handles both discrete (permutation) and continuous (real-valued) optimisation problems effectively.

Thank You

Repository: github.com/piotrek1459/biai-genetic-cartpole

```
# Run TSP
python3 -m src.task1_tsp.tsp_ga

# Run CartPole
python3 -m src.task2_cartpole.train_ga
python3 -m src.task2_cartpole.evaluate
```

Piotr Krupiński · Jeremi Szczotka